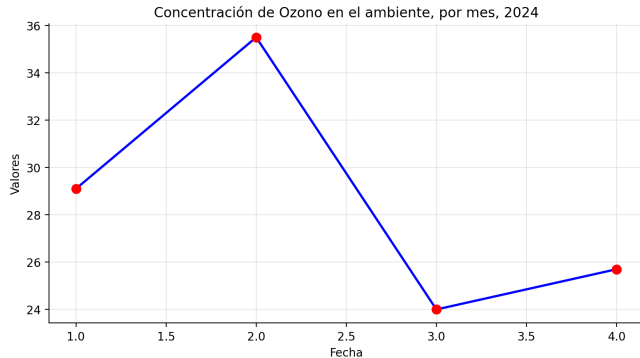


Las partes de un gráfico

Introducción a la Ciencia de Datos

Primera parte: un gráfico, una serie

El gráfico del que partimos



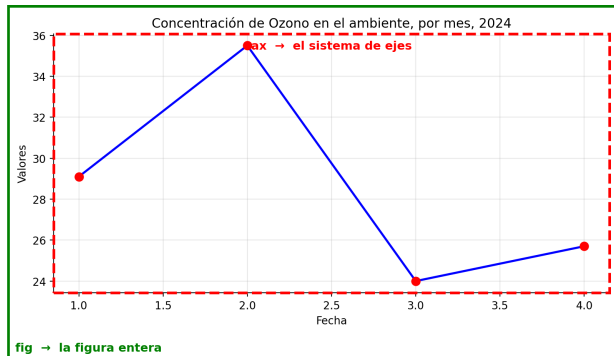
Concentración media mensual de ozono, de enero a abril. Una sola serie, cuatro observaciones.

El código que lo produce

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(ozono_mes["mes"], ozono_mes["o3_medio_mes"],
        color="blue", linewidth=2)                                # línea
ax.scatter(ozono_mes["mes"], ozono_mes["o3_medio_mes"],
           color="red", s=60, zorder=3)                          # puntos
ax.set(title="Concentración de Ozono en el ambiente, por mes, 2024",
        xlabel="Fecha", ylabel="Valores")
ax.grid(alpha=0.3)
for lado in ("top", "right"):
    ax.spines[lado].set_visible(False)
plt.tight_layout()
plt.show()
```

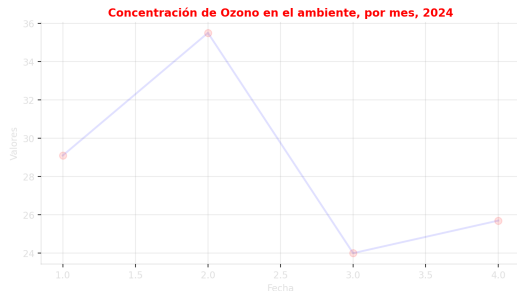
Cada línea de este código controla una parte distinta del gráfico. Las vamos a recorrer una por una.

Dos objetos, no uno



- ▶ `fig` es el lienzo completo: tamaño, resolución, márgenes, archivo de salida.
- ▶ `ax` es el sistema de ejes: donde viven los datos, las escalas y las marcas.
- ▶ Una figura puede contener varios sistemas de ejes; por eso `plt.subplots()` devuelve las dos cosas.

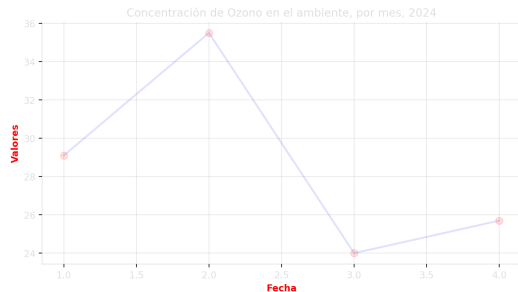
El título



`ax.set(title="Concentración de Ozono en el ambiente, por mes, 2024")`

- ▶ Dice **qué** se está midiendo, **de quién** y **cuándo**.
- ▶ Un título describe el contenido; también puede afirmar el hallazgo ("El ozono alcanza su máximo en verano"). La segunda opción exige estar seguro de lo que el gráfico muestra.

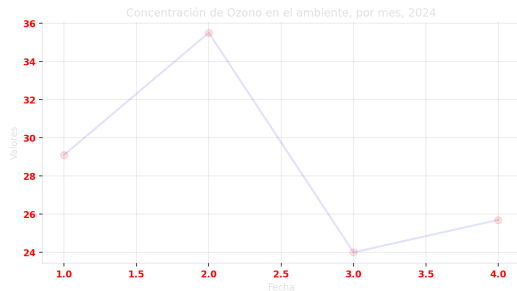
Las etiquetas de los ejes



```
ax.set(xlabel="Fecha", ylabel="Valores")
```

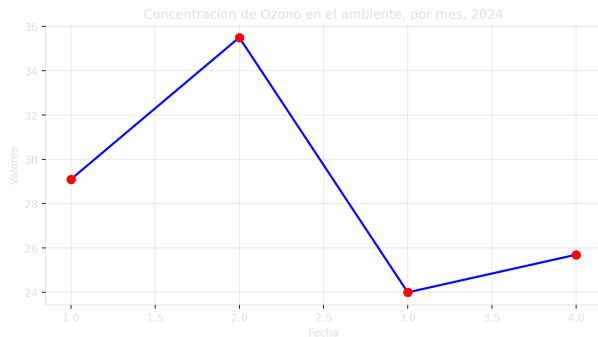
- ▶ Estas dos etiquetas son el punto débil del gráfico. “Valores” no informa nada: falta la variable y falta la unidad.
- ▶ El eje vertical debería decir qué se mide y en qué unidad; el horizontal, que la unidad de tiempo es el mes.

Las marcas y sus etiquetas



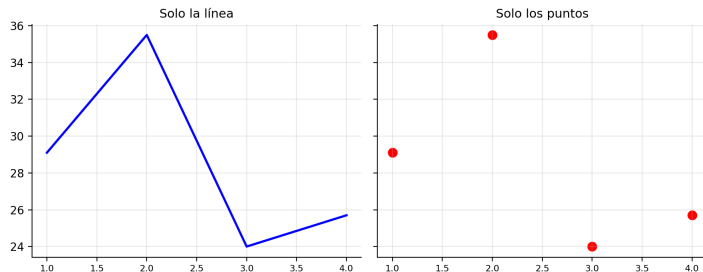
- ▶ El eje horizontal muestra marcas en 1.5, 2.5 y 3.5: meses que no existen. Pasa porque mes es numérico y matplotlib lo trata como continuo.
- ▶ Se corrige fijando las marcas a mano: `ax.set_xticks([1, 2, 3, 4])` y `ax.set_xticklabels(["Ene", ...])`.
- ▶ El eje vertical tampoco arranca en cero: admisible, pero exagera la variación.

Los datos



- ▶ Es la única parte que aporta información nueva; todo lo demás sirve para leerla.
- ▶ Cuatro pares (mes, concentración) traducidos a posiciones dentro del área de los ejes.

La geometría: dos capas superpuestas



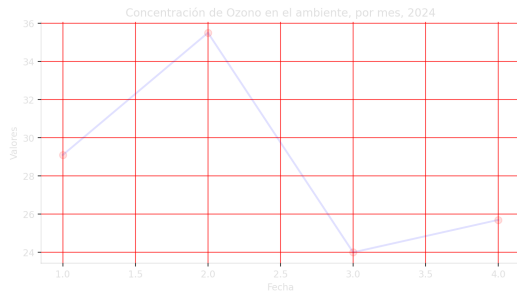
- ▶ La **línea** (`ax.plot`) afirma que hay continuidad entre meses consecutivos y dirige la lectura hacia la tendencia.
- ▶ Los **puntos** (`ax.scatter`) marcan dónde hay una medición efectiva y frenan la lectura de valores intermedios que nadie observó.
- ▶ Juntos: se ve la tendencia sin perder de vista que los datos son discretos.

Atributos estéticos

```
ax.plot(..., color="blue", linewidth=2)  
ax.scatter(..., color="red", s=60, zorder=3)
```

- ▶ color, linewidth y s no codifican ninguna variable: acá son decisiones de legibilidad, no información.
- ▶ Un atributo estético pasa a ser informativo cuando se lo asigna a una variable, por ejemplo un color por estación del año.
- ▶ zorder=3 define qué se dibuja encima. Sin eso, la línea taparía los puntos.

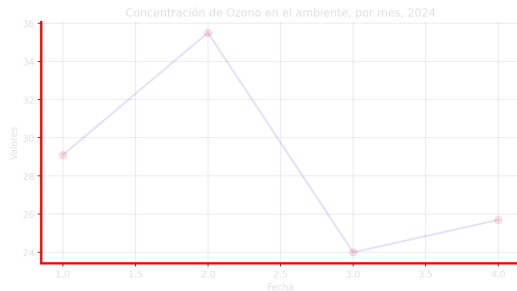
La grilla



`ax.grid(alpha=0.3)`

- ▶ Ayuda a estimar valores concretos sin recorrer el eje con la vista.
- ▶ `alpha=0.3` la deja visible pero por detrás de los datos. Una grilla opaca compite con la serie.

Los bordes



```
for lado in ("top", "right"):
    ax.spines[lado].set_visible(False)
```

- ▶ Los bordes superior y derecho no aportan referencia alguna: se sacan.
- ▶ Los bordes izquierdo e inferior sostienen las marcas, así que se quedan.
- ▶ Criterio general: quitar todo elemento que no ayude a leer un valor.

Lo que este gráfico todavía no tiene

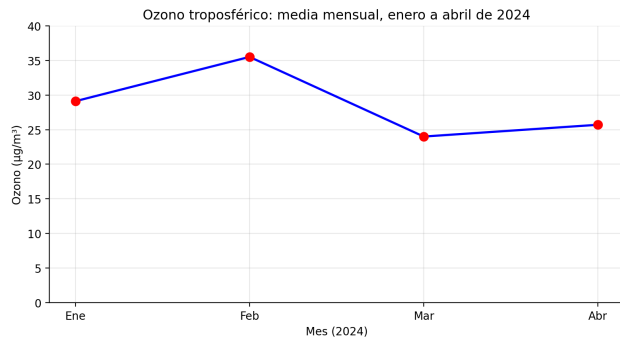
- ▶ **Unidades:** sin ellas no se puede juzgar si los valores son altos o bajos.
- ▶ **Fuente de los datos:** de dónde salen las mediciones y quién las produjo.
- ▶ **Leyenda:** acá no hace falta, porque hay una sola serie. Con dos o más pasa a ser obligatoria.
- ▶ **Nota al pie:** aclaraciones sobre cobertura, faltantes o cambios de método.

La figura es un objeto

```
fig, ax = plt.subplots(figsize=(8, 5))      # fig y ax quedan disponible  
ax.plot(...)  
fig.savefig("ozono_2024.png", dpi=300, bbox_inches="tight")  
plt.show()                                  # siempre al final
```

- ▶ `plt.subplots()` ya devuelve el objeto: `fig` es la figura y se la puede seguir modificando después de dibujarla.
- ▶ `fig.savefig()` tiene que ir antes de `plt.show()`. Con algunos backends, mostrar la figura la vacía y el archivo sale en blanco.
- ▶ En un notebook, escribir `fig` en una celda la vuelve a mostrar sin recalcular nada. Para rehacerla con un cambio: `ax.set_ylim(0, 40)` y guardar de nuevo.
- ▶ `bbox_inches="tight"` recorta los márgenes sobrantes; `dpi=300` es lo mínimo razonable para imprimir o proyectar.

Una versión corregida



Fuente: valores leídos del gráfico original.

Mismos datos, mismas geometrías. Cambian el título, las etiquetas, las marcas del eje horizontal, el rango del eje vertical y la fuente al pie.

Resumen de la primera parte

Parte	Qué controla	Dónde se toca
Figura	lienzo y tamaño	<code>figsize=</code>
Ejes	área de datos y escalas	<code>ax</code>
Título	qué, de quién, cuándo	<code>title=</code>
Etiquetas de eje	variable y unidad	<code>xlabel=</code> , <code>ylabel=</code>
Marcas	posición a valor	<code>set_xticks()</code> , <code>set_ylim()</code>
Geometría	cómo se dibujan	<code>plot()</code> , <code>scatter()</code>
Atributos	color, grosor, orden	<code>color=</code> , <code>s=</code> , <code>zorder=</code>
Grilla	lectura de valores	<code>grid()</code>
Bordes	marco del área	<code>spines[...]</code>
Guardado	archivo de salida	<code>fig.savefig()</code>

Segunda parte: más de una serie

Qué cambia cuando hay más de una serie

Con una serie, todas las decisiones eran de legibilidad. Con dos o más, aparecen decisiones de **codificación**: qué variable va al color, qué va a un panel aparte, en qué escala se lee cada eje.

- ▶ Un color por grupo: el atributo estético pasa a ser informativo.
- ▶ Superponer o apilar: mismo eje o ejes distintos.
- ▶ Varios sistemas de ejes en una figura.
- ▶ Tres geometrías y la pregunta que responde cada una.
- ▶ Escalas: logarítmica y marcas a mano.
- ▶ Color continuo y color discreto.
- ▶ Paneles: la misma pregunta, varias veces.

Los ejemplos son los de la clase 9: ozono y PM2.5 en Montevideo, enero a abril de 2024, Catálogo de Datos Abiertos.

Un color por grupo

En la primera parte, `color="blue"` era una decisión de legibilidad. Un atributo pasa a ser **informativo** cuando se lo asigna a una variable.

```
for est, d in o3_dia.groupby("estacion"):
    ax.plot(d["fecha"], d["media"], label=est)
ax.legend(title="Estación")
```

- ▶ `groupby` parte la tabla en una subtabla por estación.
- ▶ El `for` dibuja cada subtabla; `matplotlib` asigna un color distinto por llamada.
- ▶ `label=` nombra cada trazo; `legend()` los muestra.
- ▶ Con dos o más series, la leyenda deja de ser opcional.

Este patrón (`for` sobre `groupby`, `label`, `legend`) es el mismo para líneas, puntos y barras.

Superponer o apilar

Dos series se pueden poner **en el mismo eje** o **en ejes distintos**.

- ▶ **Mismo eje:** se comparan niveles. Solo si tienen la misma unidad y rangos parecidos. Ozono por estación: sí.
- ▶ **Ejes apilados:** se comparan formas. Cuando la unidad o el rango difieren. Ozono y PM2.5: apilados, con el eje horizontal compartido.

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 7), sharex=True)
ax1.plot(o3_h["fecha"], o3_h["o3"])
ax2.plot(pm_h["fecha"], pm_h["pm2_5"])
```

- ▶ `sharex=True` alinea el eje horizontal: el mismo valor cae en la misma vertical en los dos paneles.
- ▶ Poner ozono (0-120) y PM2.5 (3-50) en el mismo eje aplasta al PM2.5 y no muestra nada.

Varios sistemas de ejes

`plt.subplots(filas, columnas)` devuelve un arreglo de ejes.

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8),
                          sharex=True, sharey=True)
grupos = aire.groupby(franjas, observed=True)
for ax, (franja, d) in zip(axes.flat, grupos):
    r = d["o3"].corr(d["pm2_5"])
    ax.scatter(d["pm2_5"], d["o3"], s=8, alpha=0.4)
    ax.set_title(f"{franja}    r = {r:.2f}")
```

- ▶ `axes.flat` recorre la grilla de ejes como una lista.
- ▶ `zip` empareja cada eje con un grupo.
- ▶ `sharex, sharey`: **obligatorios** cuando los paneles se van a comparar. Con escalas distintas por panel, la vista compara formas que no son comparables.
- ▶ Las etiquetas de eje van una vez por fila y por columna, no en cada panel.

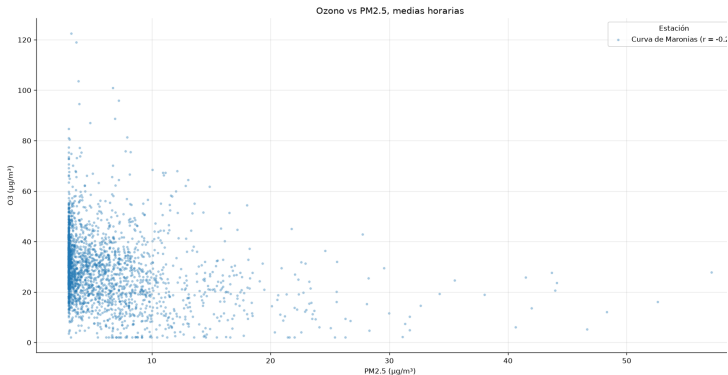
Tres geometrías, tres preguntas

Geometría	Función	Pregunta que responde
Línea	<code>ax.plot()</code>	¿Cómo cambia a lo largo de un eje ordenado?
Puntos	<code>ax.scatter()</code>	¿Cómo se relacionan dos variables?
Cajas	<code>ax.boxplot()</code>	¿Cómo se distribuye, por grupo?

- ▶ La línea afirma continuidad entre observaciones consecutivas. Solo tiene sentido si el eje horizontal es ordenado: tiempo, o una variable continua.
- ▶ Los puntos no afirman nada entre observaciones: cada uno es una medición.
- ▶ La caja resume: mediana, cuartiles, extremos. Pierde el detalle, gana comparación.

Un gráfico de barras es una línea sin continuidad: categorías, no un eje ordenado.

Dispersión: dos variables



- ▶ Cada punto es una hora. $s=6$ y $\alpha=0.3$ para que se vea dónde hay muchos.
- ▶ La pared vertical en $\text{PM}_{2.5} = 3$ no es atmósfera: es el piso del sensor.
- ▶ La leyenda quedó cortada: `figsize` demasiado ancho para el texto. Se ve antes de entregar, no después.

Densidad: cuando hay miles de puntos

Con 2.800 puntos, scatter sin cuidado es una mancha.

- ▶ `alpha=0.3`: transparencia; donde se apilan, se oscurece.
- ▶ `s=6`: tamaño del punto en puntos al cuadrado. Chico.
- ▶ `ax.set_xlim, ax.set_ylim`: recortar la zona vacía, con la advertencia de que recortar esconde extremos.
- ▶ Si aun así no se ve, `ax.hexbin(x, y, gridsize=40)` cuenta puntos por celda y colorea el conteo. Es un histograma en dos dimensiones.

El objetivo es que el lector vea **dónde está la masa** y **dónde están los pocos**. Un gráfico donde todo es igual de negro no muestra ninguna de las dos.

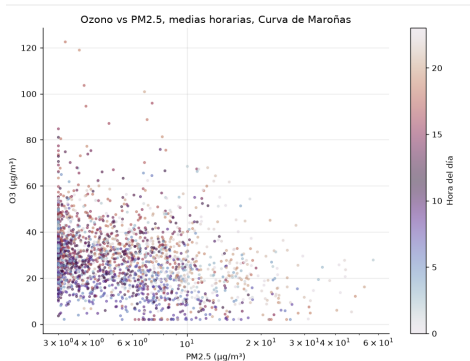
Escalas: cuándo usar logaritmo

PM2.5 va de 3 a 55, con casi todo entre 3 y 15. En escala lineal, el 80 % de los puntos ocupa el 20 % del ancho.

```
ax.set_xscale("log")
ax.set_xticks([3, 5, 10, 20, 30, 50])
ax.set_xticklabels(["3", "5", "10", "20", "30", "50"])
ax.minorticks_off()
```

- ▶ La escala logarítmica **separa lo chico y comprime lo grande**. Sirve para variables asimétricas a la derecha: concentraciones, ingresos, tamaños.
- ▶ Sin las tres líneas siguientes, las marcas salen como 3×10^0 : correctas e ilegibles. Las marcas se fijan a mano en valores redondos.
- ▶ Se usa en un eje si esa variable lo pide; no por costumbre.

Color continuo: una variable numérica

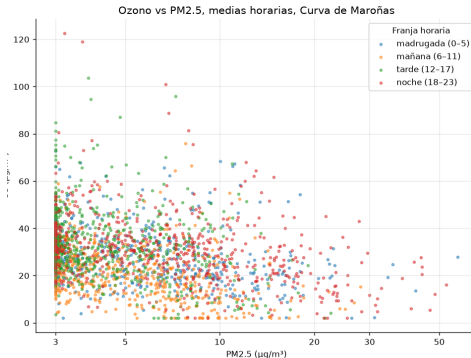


```
sc = ax.scatter(  
    x, y, s=6, alpha=0.5,  
    c=aire["fecha"].dt.hour,  
    cmap="twilight")
```

```
fig.colorbar(sc, ax=ax,  
    label="Hora del día")
```

- ▶ `c` = recibe una variable; `cmap` la traduce a color; `colorbar` es su leyenda.
- ▶ `twilight` es cíclico: 0 h y 23 h tienen el mismo color. Correcto para la hora, pero difícil de leer: el rojo de las 17 se confunde con el violeta de las 12.

Color discreto: cortar y agrupar

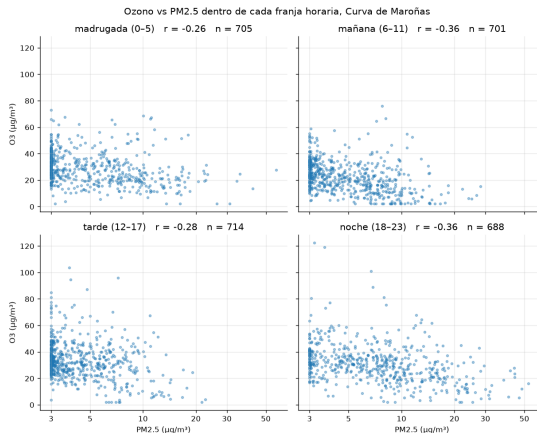


```
hora = aire["fecha"].dt.hour
frangas = pd.cut(hora,
                  bins=[-1, 5, 11, 17, 23],
                  labels=["madrugada", "mañana",
                        "tarde", "noche"])
grupos = aire.groupby(frangas,
                      observed=True)

for franja, d in grupos:
    ax.scatter(d["pm2_5"], d["o3"],
              s=8, alpha=0.5, label=franja)
```

- ▶ Cuatro colores con leyenda se leen mejor que 24 tonos con colorbar.
- ▶ **Los cortes son una decisión.** En enero el sol se pone después de las 20: la franja “noche” contiene tardes. Cambiar bins cambia lo que se ve.

Paneles: la misma pregunta, varias veces



- Un panel por franja, mismos ejes. Dentro de cada uno, r entre $-0,26$ y $-0,36$; en el gráfico sin partir, $-0,20$.
- Cuando el color mezcla demasiado, partir en paneles es la alternativa: cada panel compara horas parecidas entre sí.
- El r en el título de cada panel convierte cuatro imágenes en cuatro números comparables.

Tres figuras, tres lecturas del mismo dato

Figura	Lo que muestra	Lo que afirma
Dispersión simple Color por franja	una nube con $r = -0,2$ los extremos son de horas distintas	hay una asociación débil la hora está mezclada
Paneles por franja	$r \approx -0,3$ en cada uno	la hora diluía la asociación

- ▶ Los datos son los mismos. Cambió qué variable se codificó con color y si se partió o no.
- ▶ Una figura **sostiene una afirmación** cuando la afirmación se lee sin que haya que explicarla. Las tres son correctas; solo la tercera sostiene la última fila.
- ▶ El título puede decir la afirmación (“Dentro de cada franja, más PM2.5 va con menos ozono”). Exige estar seguro.

Antes de entregar

- ▶ **Título:** qué se mide, de quién, cuándo. O la afirmación, si está segura.
- ▶ **Unidades** en cada eje. Sin ellas no se sabe si un valor es alto o bajo.
- ▶ **Leyenda** si hay dos o más series. Legible, sin cortar.
- ▶ **Marcas** en valores que existen, con etiquetas que el lector use.
- ▶ **Fuente** al pie: de dónde salen los datos y quién los produjo.
- ▶ **Nota** sobre cobertura, faltantes o decisiones: qué estación, qué período, cómo se agregó, con qué cortes.

Y una pregunta antes de guardar: ¿qué afirmación sostiene esta figura, y se lee sin que yo esté al lado para explicarla?

Resumen de la segunda parte

Parte	Qué controla	Dónde se toca
Grupos	un color por categoría	<code>groupby + label= + legend()</code>
Paneles	varios ejes comparables	<code>subplots(..., sharex, sharey)</code>
Dispersión densa	la masa y los pocos	<code>s=, alpha=, hexbin()</code>
Escala	separar lo chico	<code>set_xscale("log") + marcas</code>
Color continuo	variable numérica	<code>c=, cmap=, colorbar()</code>
Color discreto	variable cortada	<code>pd.cut() + groupby</code>
Pie	fuentes y notas	<code>fig.text(0.01, 0.01, ...)</code>

Figuras de la primera parte: valores leídos del gráfico original (enero a abril de 2024). Figuras de la segunda parte: ozono y PM2.5, Curva de Maroñas, enero a abril de 2024, Catálogo de Datos Abiertos de Uruguay.